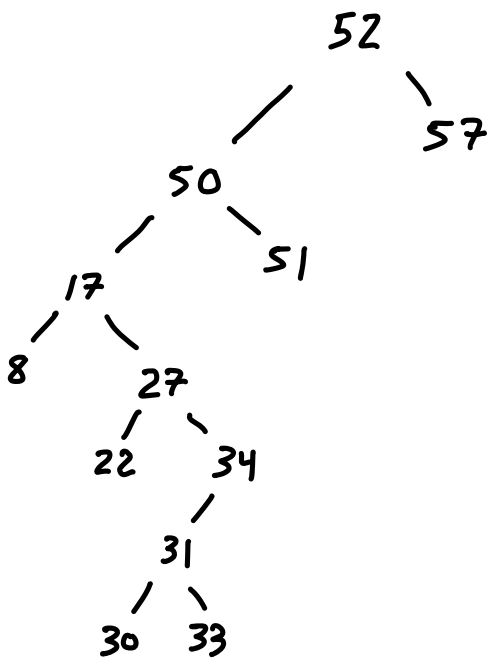
1 Basic BST Operations

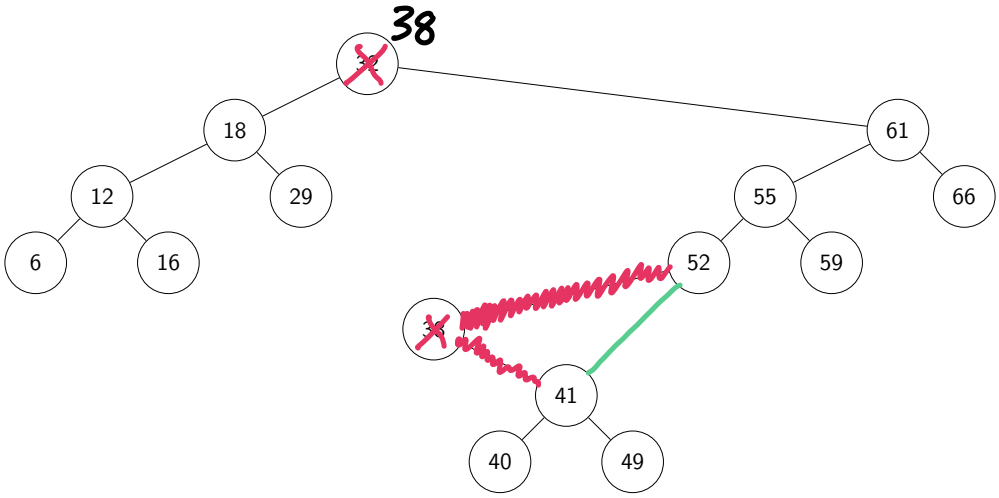
Problem 1.1. Draw the binary tree produced by inserting the following elements (**IN THE ORDER PROVIDED**) into an initially empty binary search tree.

$$52, 50, 17, 27, 51, 34, 8, 22, 31, 30, 33, 57$$

You only need to draw the final tree.
Fun note: These numbers are randomly generated, so this gives an idea of what a "random BST" looks like.

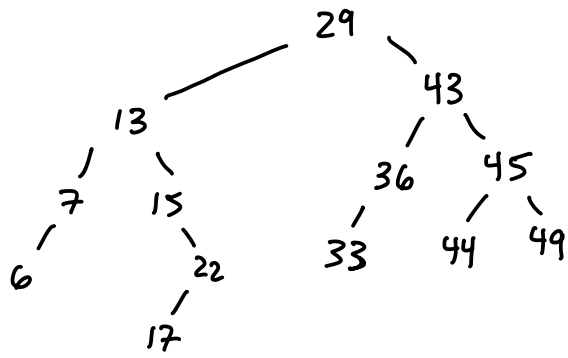


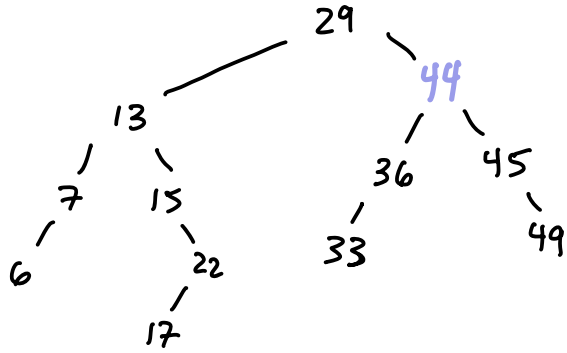
Problem 1.2. Apply the `Delete(32)` operation to the following binary search tree and draw the resulting binary tree (show your work!). State the height of the tree before deletion and after deletion.

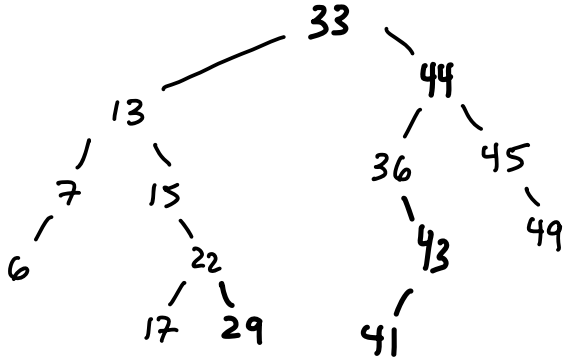


Problem 1.3. Suppose T is an initially empty BST. Perform the following sequence of operations in order. You only need to redraw the tree if your picture is unclear (you will likely need to do this a couple of times).

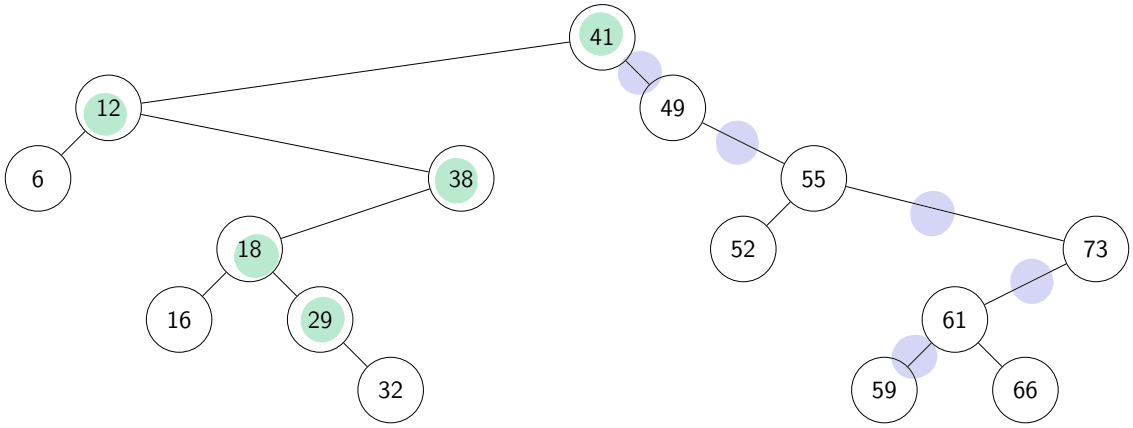
```
T.Insert(29)
T.Insert(43)
T.Insert(36)
T.Insert(13)
T.Insert(45)
T.Insert(15)
T.Insert(22)
T.Insert(33)
T.Insert(44)
T.Insert(17)
T.Insert(7)
T.Insert(6)
T.Insert(49)
T.Delete(43)
T.Delete(29)
T.Insert(43)
T.Insert(29)
T.Insert(41)
```







Problem 1.4. Consider the following Binary Search Tree



(a) What is the **root** of the tree? 41

(b) What is the **height** of this tree? 5

(c) Name one **leaf node**. 6

(d) What is the **parent** of 38? 12

(e) What is the **sibling** of 49? 12

(f) What is the **successor** of 32? 38

(g) What is the **successor** of 12? 16

(h) What is the **predecessor** of 41? 38

(i) What is the **predecessor** of 59? 55

(j) What nodes would be visited if you searched for 29? List them in the order visited.

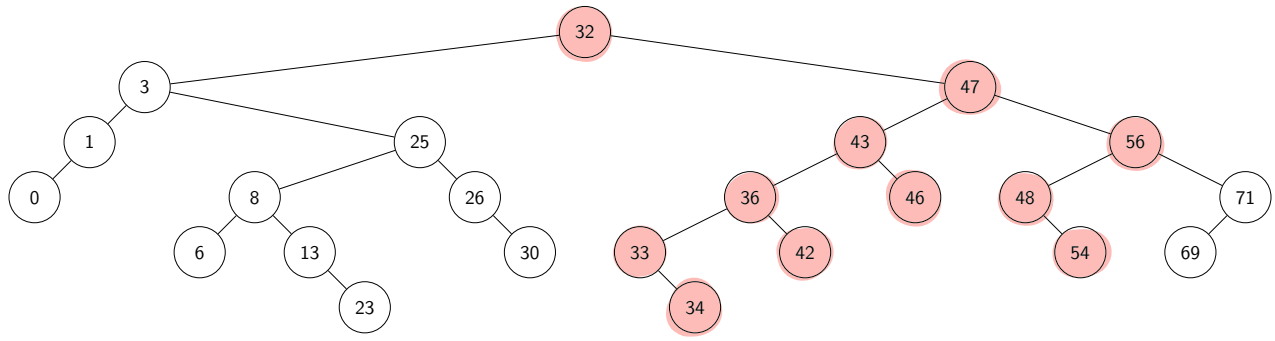
41, 12, 38, 18, 29

(k) What is one node that would be "trivial" to delete? any leaf node

(l) In general, *where* is the minimum value of a BST? left most node

(m) In general, *where* is the maximum value of a BST? right most node

Problem 1.5. Consider the following Binary Search Tree,

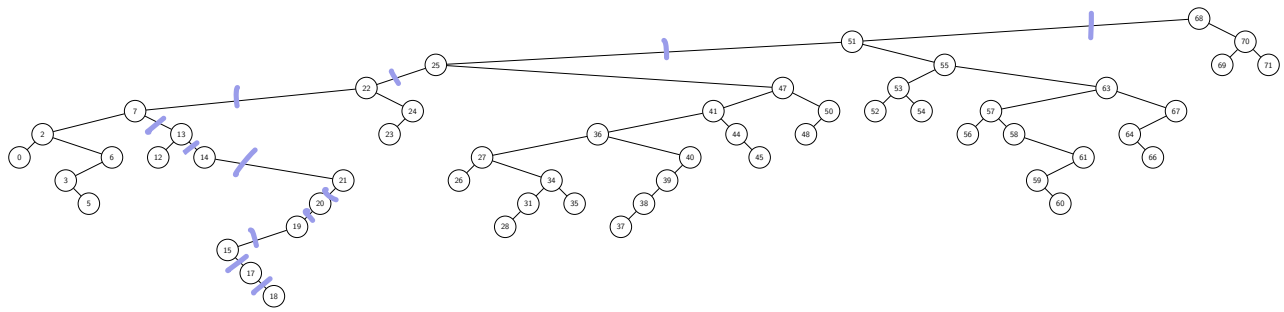


Follow the procedure outlined in the `TreeRangeReport` function in the notes to report all nodes between 35 and 52. Highlight/indicate which nodes are visited (not just printed, but **visited**) as part of this procedure.

Note: This operation is **extremely** efficient in a Binary Search Tree.

Highlighted in orange

Problem 1.6. Simply for fun, here is a BST with 55 nodes (inserted in a random order).



What are your thoughts on how **balanced** this tree is?

Not THAT bad. $h = 12$

$\log_2(55) \approx 5. \ldots$

a little more than $2 \cdot$ Best.

Problem 1.7. Consider a binary search tree where each tree node v has a field v.`size` which stores the size of the binary search tree rooted at v (size is the number of nodes in the subtree rooted at v).

(a) Write a binary search tree method called `BST.Find`(k) that returns the kth order statistic of the binary search tree. Recall that the kth order statistics is the kth smallest element of the tree. So, the 1st order statistic is the minimum, the 24th order statistic is the 24th smallest value, and so on.

Hint: Your algorithm should rely on the `size` field!

(b) Explain (in words) why your algorithm should be correct. Feel free to use pictures or test cases to support your explanation.

(c) Explain why your algorithm takes no more than $O(h)$ time.

Assume NIL.size = 0.

(d) In one line, write a BST method called `BST.Median`$()$ which returns the median in $O(h)$ time.

a) Recursive

```
function Find (T, k)
    S_L = T.left.size
    if k = S_L + 1
        return T
    elseif k < S_L + 1
        return Find (T.left, k)
    else
        return Find(T.right, k - (S_L + 1))
    endif
endfunction
```

Iterative

```
function BST.Find (k)
    x = BST.root
    S_L = 0
    while x ≠ NIL
        S_L = x.left.size
        if k = S_L + 1
            break
        elseif k < S_L + 1
            x = x.left
        else
            x = x.right
            k = k - (S_L + 1)
        endif
    endwhile
    return x
endfunction
```

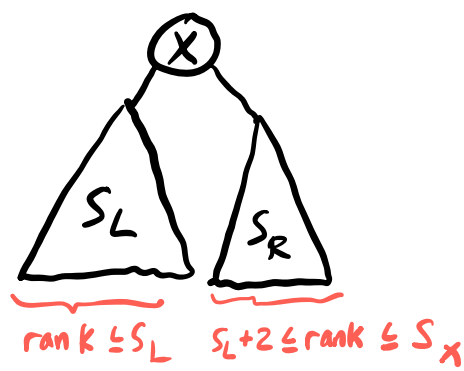
b) The correctness of this algorithm is not obvious. First, suppose x is the root of a subtree of a BST. We claim $\text{rank}(x) \equiv S_L + 1 = x.\text{left.size} + 1$. By the BST property, all nodes left of x are $< x$ and all nodes right of x are $> x$.

(assuming no duplicates)

So, in the subtree rooted at x, there are S_L values $< x$, therefore $\text{rank}(x) = S_L + 1$.

Further, since all nodes in $x.\text{left}$ are $< x$, the ranks of all of those nodes are between 1 and S_L.

Similarly, if $S_x = x.\text{size}$, the ranks of all nodes to the right of x are between $S_L + 2$ and S_x.



The strangeness appears in that traversing left and traversing right are not identical. In particular, the original problem is given a BST with n nodes, find the element x with rank k, where $k \in [1, n]$.

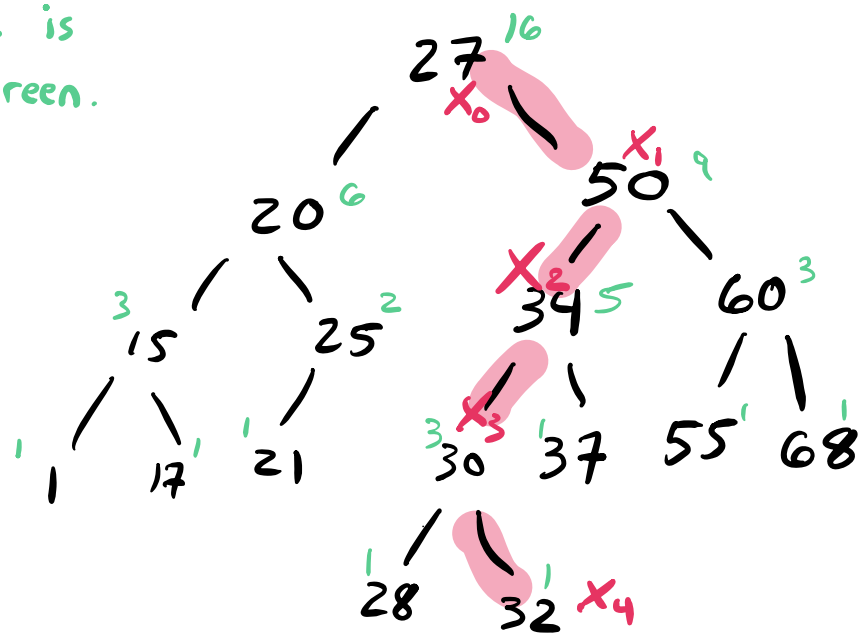
When going left, we seek the node x with rank $k \in [1, S_L]$. Further, we only go left when $k < S_L + 1$ so we are searching correctly. Finally, searching $[1, n]$ is **EXACTLY** the same as $[1, S_L]$.

When going right we seek the node x with rank $k \in [S_L + 2, n]$. Note that this interval is of a different form; if we shift the interval down by $S_L + 1$,

then we can search for the element of rank $k - (s_L + 1) \in [1, n - (s_L + 1)]$. This is now the exact form we desire.

Let's explore with an example!

size is in green.



Call BST.Find(10).

x = Root (27)

$s_L = 6$.

$10 > 6+1$ so

$x = x.\text{right}$ (50)

NOTICE: For the subtree rooted at 50, the 10th smallest node is <u>NOT</u> 32. k must be changed! 32 is the 3rd smallest node.

$k = 10 - 7 = 3$.

$s_L = 5$

$3 < 6$ so $x = x.\text{left}$ (34)

NOTICE: For the subtree rooted at 34, the 3rd smallest node <u>IS</u> 32!

$s_L = 3$

$3 < 4$ so $x = x.\text{left}$ (30)

$s_L = 1$

$3 > 2$ so $x = x.\text{right}$ (32)

$s_L = 0$

$1 = 1$ so return 32.

c) Algo. only goes down the tree and each op. takes $O(1)$ time, so $O(h)$.

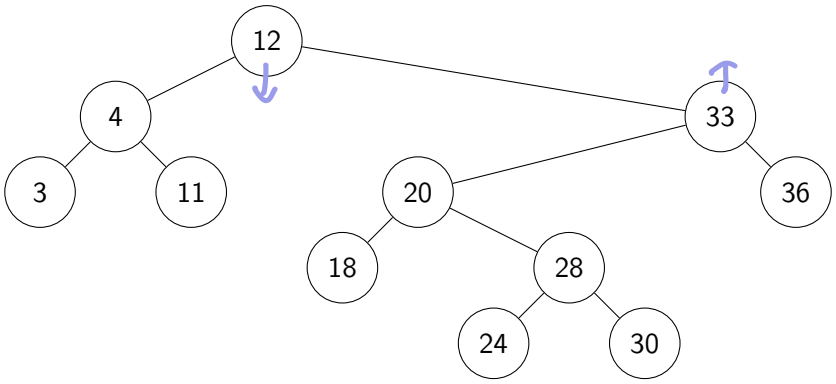
d) function BST.Median()

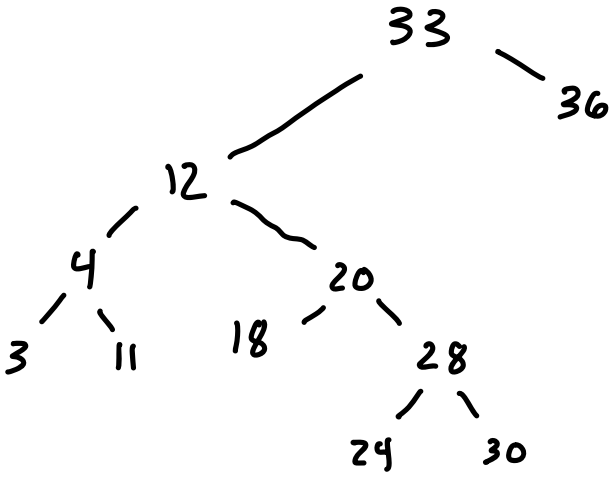
return BST.Find$\left(\left\lceil \frac{\text{BST.size}}{2} \right\rceil\right)$

endfunction

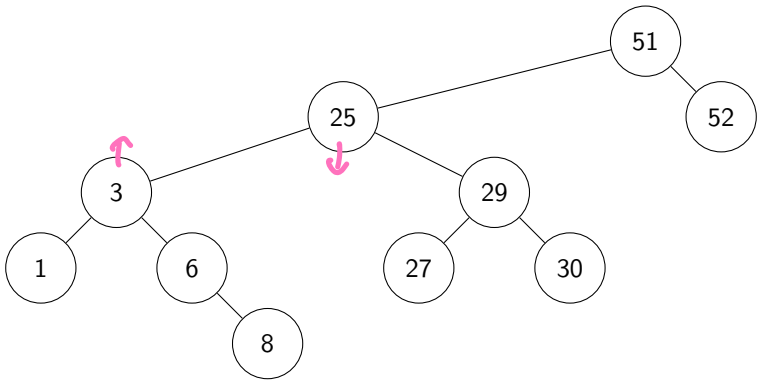
2 BST Rotations and Balancing

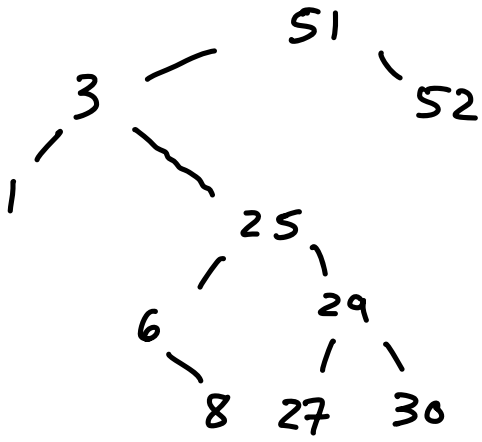
Problem 2.1. Apply the `LeftRotate(12)` operation to the following binary search tree and draw the resulting binary tree. It may help to indicate which node(s) "go down" and which node(s) "come up".





Problem 2.2. Apply the `RightRotate(25)` operation to the following binary search tree and draw the resulting binary tree. It may help to indicate which node(s) "go down" and which node(s) "come up".

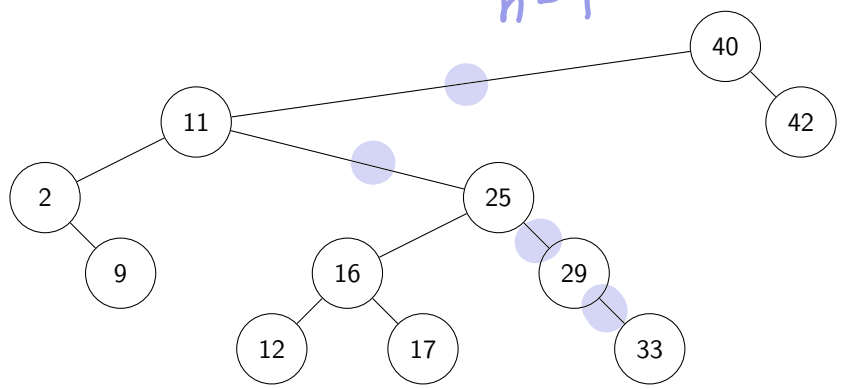




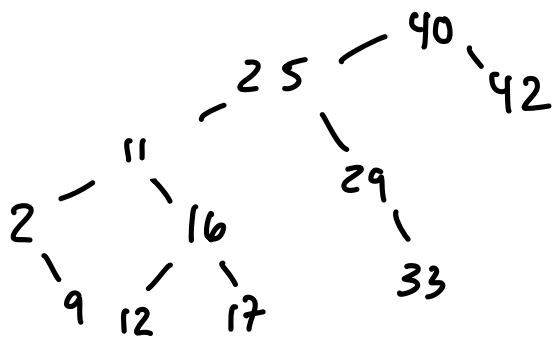
Problem 2.3. Determine a set of rotations that will reduce the height of the following binary search tree by one. Give the rotations **and** draw the results of applying each rotation to the tree.

Hint: This may require more than one rotation!

h=4

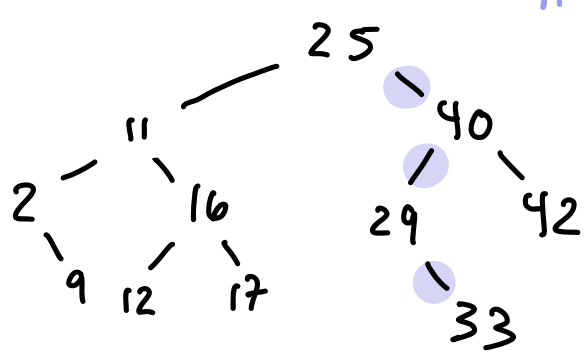


LR(11)



RR(40)

h=3

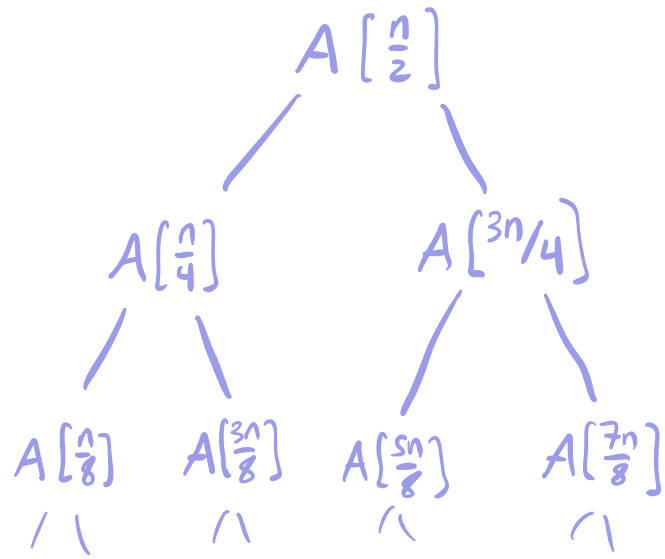


Problem 2.4. Write a function that takes an (unsorted) array as input, and returns a binary search tree constructed from the elements of the array such that the height of the binary search tree is at most $2\log_2(n)$. Worded more naturally: given an unsorted array, write a method for constructing a binary search tree that is AS BALANCED AS POSSIBLE. You may assume this function has access to the nodes and pointers of the BST. You are allowed to modify the array in any way to make your algorithm easier/more efficient.
I recommend **not** using rotations for this problem.

```
function BuildBalancedBST(A[], n)
    Sort(A[], n)
    Let T = empty BST
    Build(A[], 1, n, T)
    return T
end function

function Build(A, i, j, T)
    if i ≤ j
        mid = ⌊(i+j)/2⌋
        T.Insert(A[mid])
        Build(A[], i, mid-1, T)
        Build(A[], mid+1, j, T)
    endif
```

we want



Problem 2.5 (Optional). Determine the asymptotic running time of your algorithm from the previous problem. Fully justify your answer.

```
function BuildBalancedBST(A[], n)
    Sort(A[], n)                O(n log(n))
    Let T = empty BST           O(1)
    Build(A[], 1, n, T)
    return T
endfunction

function Build(A, i, j, T)
  if i ≤ j
    mid = ⌊(i+j)/2⌋
    T.Insert(A[mid])            O(log(n))
    Build(A[], i, mid-1, T)
    Build(A[], mid+1, j, T)
  endif
```

n nodes are inserted and since we insert s.t. height is $O(\log(n))$, we know Build $\in O(n\log(n))$

So, $T(n) \in \Theta(n\log(n))$.

<u>Note</u>: a faster alg. may exist for Build since the required steps are:

1. $\forall$ values $A[i]$, make a node x_i.
2. Assign x_i.left, x_i.right, and x_i.parent.

$\Big\}\ \Theta(n)$.

We are too lazy to code this.